

SMART_AO V8 — APP-01

Contrats Pydantic des commandes, réponses et erreurs — Premier slice

Version : 1.0

Statut : contrat d’ application à implémenter avant les premiers endpoints FastAPI

Auteur : Manus AI

Périmètre : commandes, résultats, erreurs et références pour Case , Consultation , DceVersion et Decision

Dépendances normatives : DOMAIN-01 v1.1, DOMAIN-03 v1.0 et conventions d’ idempotence V8

1. Objet et règle de dérivation

APP-01 transforme les commandes et états déjà décidés dans DOMAIN-03 en modèles Pydantic v2. Il n’ ajoute aucune transition métier, aucun écran et aucune règle de persistance. Un modèle qui ne peut pas être rattaché à une ligne de DOMAIN-03 est hors périmètre et ne doit pas être créé.

Décision APP-01 : les modèles Pydantic valident la forme, les formats, les bornes et les références déclarées d’ une intention. Les handlers de domaine restent seuls responsables de l’ autorisation finale, de la vérification tenant, de la concurrence, des invariants, de la transition et de l’ émission d’ événements.

Niveau	APP-01 définit	APP-01 ne définit pas
Entrée API	Corps de commande, types, champs requis, formats, limites, clés de rejouabilité.	Tenant, identité, rôle ou permissions de confiance venant du navigateur.
Application	Réponse de succès, suivi, rejet et erreur typée.	Mutation ORM, transaction SQL ou outbox concrète.
Domaine	Référence explicite vers préconditions/invariants DOMAIN-03.	Nouvel état ou nouvelle transition.
Infrastructure	Contrat d’ erreur et d’ idempotence consommé par FastAPI/PostgreSQL.	Schéma de table, migration Alembic ou mécanisme de worker.

2. Conventions Pydantic V8

2.1. Conventions de nommage et de validation

Sujet	Contrat
Langage de transport	JSON en <code>snake_case</code> ; le code Python garde les mêmes noms.
Date/heure	<code>datetime</code> timezone-aware ISO-8601. Une date métier seule emploie <code>date</code> .
Identifiants	<code>UUID</code> non nul ; aucun ID entier exposé comme référence métier.
Révisions	Entier <code>>= 0</code> ; zéro correspond à la première révision persistée.
Texte	Espaces de bord supprimés, chaînes vides rejetées si le champ est requis.
Champs inconnus	Refusés systématiquement par <code>extra="forbid"</code> .
Montants	Hors périmètre APP-01. Aucun <code>float</code> , prix, marge ou devis privé ne figure dans ces contrats.
Tenant / identité	Résolus uniquement côté serveur, donc absents des corps de requêtes publiques.
Objet versionné	Une référence de contenu inclut toujours <code>aggregate_id</code> et, lorsque le contenu est sensible à la fraîcheur, <code>aggregate_revision</code> ou <code>content_hash</code> .

2.2. Base de modèles Python à créer telle quelle

Python

```
from __future__ import annotations

from datetime import date, datetime
from typing import Annotated, Generic, Literal, TypeVar
```

```

from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field, HttpUrl, field_validator

NonEmptyStr = Annotated[str, Field(min_length=1, max_length=4_000)]
ShortLabel = Annotated[str, Field(min_length=1, max_length=240)]
Sha256Hex = Annotated[str, Field(pattern=r"^[a-f0-9]{64}$")]
Revision = Annotated[int, Field(ge=0)]

class ApiModel(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        str_strip_whitespace=True,
        validate_assignment=True,
    )

class ServerResolvedContext(ApiModel):
    """Construit côté serveur ; jamais désérialisé depuis le navigateur."""

    tenant_id: UUID
    actor_id: UUID
    actor_kind: Literal["PATRON", "DELEGATAIRE", "COLLABORATEUR", "SYSTEM"]
    received_at: datetime

```

`ServerResolvedContext` est un contrat interne d'application. Il peut être enrichi par des droits calculés, mais le client ne peut jamais le fournir.

3. Primitives communes de commande

3.1. Métadonnées de toute intention publique

Python

```

class ExpectedAggregateRevision(ApiModel):
    aggregate_type: Literal["AFF", "DCE", "DEC"]
    aggregate_id: UUID
    expected_revision: Revision

class PublicCommandMeta(ApiModel):
    command_id: UUID
    idempotency_key: UUID

```

```
correlation_id: UUID | None = None
expected_reference_revisions: list[ExpectedAggregateRevision] = Field(defau
```

```
class ExistingAggregateCommandMeta(PublicCommandMeta):
    expected_revision: Revision
```

```
PayloadT = TypeVar("PayloadT", bound=ApiModel)
```

```
class CommandRequest(ApiModel, Generic[PayloadT]):
    meta: PublicCommandMeta
    payload: PayloadT
```

```
class ExistingAggregateCommandRequest(ApiModel, Generic[PayloadT]):
    meta: ExistingAggregateCommandMeta
    payload: PayloadT
```

Le `command_type` ne vient pas du client dans le payload : il est fixé par la route FastAPI et par le modèle de requête attendu. Par exemple, la route `POST /api/v1/cases` accepte exclusivement `CreateCaseRequest` et assigne `command_type = "CreateCase"` côté serveur.

3.2. Métadonnées internes pour commandes aval

Python

```
class InternalCommandMeta(ExistingAggregateCommandMeta):
    caused_by_event_id: UUID
    correlation_id: UUID
```

`InternalCommandMeta` est réservé aux Process Managers. Il ne doit pas être exposé dans une route accessible au navigateur. Toute commande aval doit porter le même `correlation_id` que l'événement déclencheur et son `caused_by_event_id`.

3.3. Références typées communes

Python

```
class AggregateRef(ApiModel):
    aggregate_type: Literal["AFF", "DCE", "DEC"]
    aggregate_id: UUID
    aggregate_revision: Revision
```

```

class VersionedReference(ApiModel):
    aggregate_type: Literal["AFF", "DCE", "DEC", "PRF", "PRX", "DEP", "SNP", "A
    aggregate_id: UUID
    aggregate_revision: Revision | None = None
    content_hash: Sha256Hex | None = None

class SourceLocator(ApiModel):
    locator_kind: Literal["FILE", "PAGE", "TEXT_SPAN", "TABLE_CELL", "PLAN_ZONE
    page_number: int | None = Field(default=None, ge=1)
    text_start: int | None = Field(default=None, ge=0)
    text_end: int | None = Field(default=None, ge=0)
    table_reference: str | None = Field(default=None, max_length=240)
    note: str | None = Field(default=None, max_length=1_000)

    @field_validator("text_end")
    @classmethod
    def text_span_must_be_ordered(cls, value: int | None, info):
        start = info.data.get("text_start")
        if value is not None and start is not None and value < start:
            raise ValueError("text_end must be greater than or equal to text_st
        return value

```

Une `VersionedReference` indique seulement ce que le client a lu. Le handler contrôle que la référence existe, appartient au tenant, est autorisée, correspond au sujet de la commande et est encore utilisable.

4. Contrats de réponse communs

4.1. Résultat d' une commande durable

Python

```

class ProjectionStatus(ApiModel):
    status: Literal["CURRENT", "REFRESH_PENDING", "PARTIAL"]
    refreshed_at: datetime | None = None
    tracking_correlation_id: UUID | None = None

class NavigationHint(ApiModel):
    view: str
    primary_aggregate_id: UUID | None = None
    secondary_aggregate_id: UUID | None = None

```

```

class CommandSuccess(ApiModel):
    status: Literal["SUCCEEDED"] = "SUCCEEDED"
    command_id: UUID
    idempotency_key: UUID
    result_code: str
    aggregate_refs: list[AggregateRef] = Field(min_length=1)
    event_ids: list[UUID] = Field(default_factory=list)
    projection: ProjectionStatus
    navigation: NavigationHint | None = None
    replayed: bool = False

class CommandProcessing(ApiModel):
    status: Literal["PROCESSING"] = "PROCESSING"
    command_id: UUID
    idempotency_key: UUID
    correlation_id: UUID
    result_code: Literal["IDEMPOTENCY_IN_PROGRESS", "PROCESSING_AFTER_COMMIT"]
    retry_after_seconds: int = Field(ge=1, le=120)
    tracking_url: str | None = None

```

Une réponse `SUCCEEDED` signifie que la mutation du root, l'événement métier, l'outbox applicable et le résultat idempotent ont été commités ensemble. `REFRESH_PENDING` n'annule jamais le succès ; il signale seulement qu'une projection de lecture n'est pas encore à jour.

4.2. Rejet métier et problème de validation

Python

```

class ResourceConflict(ApiModel):
    aggregate_type: str
    aggregate_id: UUID | None = None
    current_revision: Revision | None = None
    reason: str | None = Field(default=None, max_length=500)

class FieldViolation(ApiModel):
    field_path: str
    code: str
    message: str

class CommandRejected(ApiModel):
    status: Literal["REJECTED"] = "REJECTED"
    command_id: UUID | None = None
    code: str

```

```

message: str
next_action: str | None = None
changed_resources: list[ResourceConflict] = Field(default_factory=list)
violations: list[FieldViolation] = Field(default_factory=list)
correlation_id: UUID | None = None

```

Réponse	HTTP	Usage
CommandSuccess	200 ou 201	Création, mutation ou replay d' un succès terminal.
CommandProcessing	202	Même intention durable, toujours en cours ou étape aval en traitement.
CommandRejected de forme	422	Type, format, champ requis ou contrainte Pydantic invalide.
CommandRejected de droit / existence	403 ou 404 neutre	Le serveur ne révèle pas une ressource inter-tenant.
CommandRejected de conflit métier	409	Révision, contexte, doublon fonctionnel ou clé d' idempotence en conflit.
CommandRejected de précondition	422	Invariant ou étape métier non satisfaite.

5. Catalogue global des codes d' erreur du premier slice

Code	HTTP	Quand il est retourné
VALIDATION_ERROR	422	Modèle Pydantic invalide ou champ inconnu.
NOT_FOUND_OR_FORBIDDEN	404	Ressource absente ou inaccessible sans révélation inter-tenant.
PATRON_AUTHORIZATION_REQUIRE D	403	Finalisation Decision tentée hors autorité patron/délégation explicite.
IDEMPOTENCY_KEY_REQUIRED	400	Métadonnée durable manquante.

IDEMPOTENCY_KEY_REUSED	409	Même clé avec empreinte différente.
IDEMPOTENCY_IN_PROGRESS	202	Même intention même empreinte en cours.
VERSION_CONFLICT	409	Root a évolué depuis la lecture.
STALE_CONTEXT	409	Référence critique ou fingerprint de décision devenu différent.
DUPLICATE_FUNCTIONAL_IDENTITY	409	Aggregate métier identique déjà actif.
CASE_SCOPE_AMBIGUOUS	422	Périmètre Case absent, mélangé ou non justifié.
CASE_LIFECYCLE_FORBIDS_ACTION	422	Commande incompatible avec lifecycle/stage Case.
DCE_VERSION_NOT_APPLICABLE	422	DceVersion retirée, autre consultation ou scope incompatible.
DOCUMENT_ORIGINAL_IMMUTABLE	422	Tentative de modifier l' original admis.
DCE_VERSION_DUPLICATE	409	Même corpus hash avec contenu de commande incompatible.
DCE_VERSION_UNUSABLE	422	Action analytique impossible sur version inutilisable.
SOURCE_LOCATION_REQUIRED	422	SourceStatement sans document ou localisation.
DECISION_CONTEXT_INCOMPLETE	422	Contexte requis pour le DecisionType incomplet.
DECISION_CONTEXT_FINGERPRINT_MISMATCH	409	Empreinte affichée différente du contexte courant.
CONDITION_OWNER_REQUIRED	422	Go conditionnel sans responsable.
CONDITION_CONSEQUENCE_REQUIRED	422	Go conditionnel sans conséquence de non-réalisation.

DECISION_ALREADY_FINALIZED	422	Tentative de modifier outcome/contexte finalisé.
DECISION_CONDITION_NOT_OPEN	422	Condition introuvable, déjà close ou Decision non conditionnelle.

Les messages doivent expliquer l’ action corrective, mais ne peuvent exposer ni contenu financier ni ressource hors tenant.

Partie I — Contrats Case

6. Sous-modèles Case

Python

```
class CaseScopeInput(ApiModel):
    scope_kind: Literal[
        "SINGLE_LOT", "MULTI_LOT", "TRANCHE", "VARIANT", "CUSTOM_SOURCED_SCOPE"
    ]
    lot_numbers: list[ShortLabel] = Field(default_factory=list, max_length=30)
    tranche_reference: str | None = Field(default=None, max_length=240)
    variant_reference: str | None = Field(default=None, max_length=240)
    source_justification: NonEmptyStr | None = None

class CaseOriginInput(ApiModel):
    origin_kind: Literal["OPPORTUNITY", "MANUAL", "IMPORT", "CUSTOMER_REQUEST"]
    origin_reference_id: UUID | None = None
    rationale: NonEmptyStr | None = None

class CaseStageTarget(ApiModel):
    commercial_stage: Literal[
        "INTAKE", "ANALYSIS", "AWAITING_DECISION", "OFFER_PREPARATION",
        "READY_FOR_PRICING", "PRICING", "READY_FOR_FINAL_CONTROL",
        "READY_FOR_SUBMISSION", "SUBMITTED", "OUTCOME_KNOWN", "AWARDED", "EXECU
    ]

class CaseOutcomeInput(ApiModel):
    outcome: Literal["AWARDED", "LOST", "WITHDRAWN", "NO_FOLLOW_UP"]
```

```
observed_at: datetime
provenance: NonEmptyStr
source_reference: VersionedReference | None = None
```

Validation Pydantic locale : SINGLE_LOT exige exactement un lot_numbers ; MULTI_LOT exige au moins deux lots; TRANCHE , VARIANT et CUSTOM_SOURCED_SCOPE exigent leur référence/motif. La compatibilité réelle avec la Consultation est une règle de handler.

7. Commandes Case

command_type / modèle de requête	payload Pydantic obligatoire	meta	Résultat de succès
CreateCaseRequest / CreateCase	consultation_id nullable uniquement si origine manuelle, scope: CaseScopeInput , origin: CaseOriginInput , title: ShortLabel , object_description: NonEmptyStr .	PublicCommandMeta .	CASE_CREATED , ref Af rev. 0, navigation CASE_OVERVIEW .
RegisterCaseConsultationLi nkRequest	case_id , consultation_id , consultation_revision , scope: CaseScopeInput , rationale .	Existing Case revision.	CASE_CONSULTATION_L KED .
SetApplicableDceVersionRe quest	case_id , dce_version: VersionedReference , consultation_id , rationale .	Existing Case revision + external expected refs.	CASE_APPLICABLE_DCE RSION_SET .
StartCaseAnalysisRequest	case_id , starting_source: VersionedReference , note optional.	Existing Case revision.	CASE_ANALYSIS_START
RecordDecisionReadinessR equest	case_id , `readiness: NOT_READY	READY_WITH_UNKNOW NS	READY , assessment_referenc unknowns: list[NonEmptyStr] , assessed_at` .
MoveCaseToDecisionReque st	case_id , readiness_reference , note optional.	Existing Case revision + DCE expected ref.	CASE_READY_FOR_DECI N .

StartOfferPreparationRequest	Interne seulement : case_id , decision_ref .	InternalCommandMeta .	CASE_OFFER_PREPARATION_STARTED .
MoveCaseToPricingRequest	case_id , decision_ref , preparation_transmission_ref .	Existing Case revision + refs.	CASE_READY_FOR_PRICING .
StartCasePricingRequest	case_id , note optional.	Existing Case revision.	CASE_PRICING_STARTED
MarkCaseReadyForFinalControlRequest	case_id , official_pricing_version_ref , applicable_dce_ref .	Existing Case revision + refs.	CASE_READY_FOR_FINAL_CONTROL .
MarkCaseReadyForSubmissionRequest	case_id , submission_package_ref , decision_ref .	Existing Case revision + refs.	CASE_READY_FOR_SUBMISSION .
RecordCaseSubmissionDeclaredRequest	case_id , submission_declaration_ref , declared_at .	Existing Case revision.	CASE_SUBMISSION_DECLARED .
RecordCaseOutcomeRequest	case_id , outcome: CaseOutcomeInput .	Existing Case revision.	CASE_OUTCOME_RECORD .
EnterExecutionRequest	case_id , award_reference , entered_at .	Existing Case revision.	CASE_EXECUTION_ENDED .
ReturnCaseToPreparationRequest	case_id , `reason_kind`: PATRON_RETURN	DCE_IMPACT	INCONSISTENCY
StopCaseRequest	case_id , reason: NonEmptyStr , stopped_at optional.	Existing Case revision.	CASE_STOPPED .
ReopenCaseWithPatronDecisionRequest	case_id , reopening_decision_ref , target: CaseStageTarget , rationale .	Existing Case revision + decision ref.	CASE_REOPENED .
ArchiveCaseRequest	case_id , reason: NonEmptyStr ,	Existing Case revision.	CASE_ARCHIVED .

	override_open_actions_reason optional.		
RestoreArchivedCaseRequest	case_id , target: CaseStageTarget , reason: NonEmptyStr .	Existing Case revision.	CASE_RESTORED .

Chaque modèle de payload ci-dessus est une classe Pydantic distincte dans le code. Les champs de référence utilisent `VersionedReference` , jamais un objet ORM ou une charge d' un autre bounded context.

Partie II — Contrats Consultation et DceVersion

8. Sous-modèles Consultation et DCE

Python

```
class BuyerIdentityInput(ApiModel):
    legal_name: ShortLabel
    normalized_identififier: str | None = Field(default=None, max_length=120)

class ConsultationLotInput(ApiModel):
    lot_number: ShortLabel
    label: ShortLabel
    source_reference: str | None = Field(default=None, max_length=240)

class ConsultationTrancheInput(ApiModel):
    tranche_reference: ShortLabel
    tranche_kind: Literal["FIRM", "OPTIONAL", "CONDITIONAL", "OTHER"]
    label: ShortLabel
    source_reference: str | None = Field(default=None, max_length=240)

class DceDocumentInput(ApiModel):
    storage_object_id: UUID
    original_filename: ShortLabel
    media_type: ShortLabel
    byte_size: int = Field(gt=0, le=2_000_000_000)
```

```

sha256: Sha256Hex
received_from: Literal["BUYER_PLATFORM", "EMAIL", "MANUAL_UPLOAD", "RECTIFI

```

```

class DceProvenanceInput(ApiModel):
    source_channel: Literal["BUYER_PLATFORM", "EMAIL", "MANUAL_UPLOAD", "RECTIF
    source_url: HttpUrl | None = None
    source_reference: str | None = Field(default=None, max_length=240)
    source_received_at: datetime | None = None

```

```

class SourceStatementInput(ApiModel):
    dce_document_id: UUID
    locator: SourceLocator
    excerpt: NonEmptyStr
    source_language: str = Field(default="fr", min_length=2, max_length=12)
    extraction_origin: Literal["HUMAN", "OCR_CONFIRMED", "PARSER_CONFIRMED"]

```

`storage_object_id` est un identifiant MinIO interne déjà admis par un endpoint d'upload séparé. APP-01 ne transporte jamais un document binaire dans une commande JSON.

9. Commandes Consultation

command_type / modèle	payload obligatoire	meta	Résultat de succès	Erreurs principales
CreateConsultation Request / CreateConsultation	buyer: BuyerIdentityInput , external_reference nullable, object: ShortLabel , location nullable, provenance: DceProvenanceInput .	PublicCommandMeta .	CONSULTATION_CREATED , ref DCE rev. 0.	DUPLICATE_FUNCTIONAL_IDENTITY .
RegisterConsultationLotRequest	consultation_id , lot: ConsultationLotInput .	Existing Consultation revision.	CONSULTATION_LOT_REGISTERED .	VERSION_CONFLICT , DUPLICATE_FUNCTIONAL_IDENTITY .
RegisterConsultationTrancheRequest	consultation_id , tranche: ConsultationTrancheInput .	Existing Consultation revision.	CONSULTATION_TRANCHE_REGISTERED .	VERSION_CONFLICT , DUPLICATE_FUNCTIONAL_IDENTITY .

CorrectConsultationMetadataRequest	consultation_id , field_name , new_value , provenance: DceProvenanceInput , rationale .	Existing Consultation revision.	CONSULTATION_METADATA_CORRECTED .	VERSION_CONFLICT , BUSINESS_RULE_VIOLATION .
CloseConsultationRequest	consultation_id , `reason_kind: DEADLINE_EXPIRED	AWARDED	WITHDRAWN	OTHER , reason: NonEmptyStr , source_ref` optional.
ArchiveConsultationRequest	consultation_id , reason: NonEmptyStr .	Existing Consultation revision.	CONSULTATION_ARCHIVED .	VERSION_CONFLICT .

10. Commandes DceVersion

command_type / modèle	payload obligatoire	meta	Résultat de succès	Erreurs principales
RegisterDceVersion Request / RegisterDceVersion	consultation_id , consultation_revision , documents: list[DceDocumentInput] min. 1, corpus_hash: Sha256Hex , provenance: DceProvenanceInput , predecessor_dce_version_id optional.	PublicCommandMeta + reference Consultation.	DCE_VERSION_REGISTERED , ref DCE version rev. 0, integrity/readiness.	CONSULTATION_REQUIRED , DCE_CORPUS_HASH_REQUIRED , DCE_VERSION_DUPLICATE .
RegisterSupersedingDceVersionRequest	Même champs que RegisterDceVersion + predecessor_dce_version_id , rectification_reason: NonEmptyStr .	PublicCommandMeta + predecessor ref revision.	DCE_VERSION_SUPERSEDING_REGISTERED ; processus aval éventuel.	DCE_VERSION_DUPLICATE , STALE_CONTEXT .

ConfirmDocumentClassificationRequest	dce_version_id , dce_document_id , `document_class` : RC	CCAP	AE	CCTP
CorrectDocumentClassificationRequest	dce_version_id , dce_document_id , previous_classification_id , document_class , rationale: NonEmptyStr .	Existing DceVersion revision.	DCE_DOCUMENT_CLASSIFICATION_CORRECTED .	DOCUMENT_NOT_IN_VERSION , VERSION_CONFLICT .
DeclareMissingDocumentRequest	dce_version_id , expected_document_family , `expectation_source` : SourceStatementInput	VersionedReference , reason: NonEmptyStr` .	Existing DceVersion revision.	DCE_DOCUMENT_MISSING_DECLARED .
ReportDocumentIntegrityIssueRequest	dce_version_id , dce_document_id , `issue_kind` : CORRUPTED	UNREADABLE	INCOMPLETE	PASSWORD_PROTECTED
RegisterSourceStatementRequest	dce_version_id , statement: SourceStatementInput .	Existing DceVersion revision.	DCE_SOURCE_STATEMENT_REGISTERED .	SOURCE_LOCATION_REQUIRED , DCE_VERSION_UNUSABLE .
MarkDceVersionReadyForAnalysisRequest	dce_version_id , accept_partial_integrity: bool , known_limitations: list[NonEmptyStr] .	Existing DceVersion revision.	DCE_VERSION_READY_FOR_ANALYSIS .	DCE_VERSION_UNUSABLE , BUSINESS_RULE_VIOLATION .
MarkDceVersionReviewRequiredRequest	Interne ou patron explicitement habilité : dce_version_id , reason_kind , reason , trigger_ref .	Existing/Internal meta.	DCE_VERSION_REVIEW_REQUIRED .	VERSION_CONFLICT .
WithdrawDceVersionRequest	dce_version_id , withdrawal_source:	Existing DceVersion	DCE_VERSION_WITHDRAWN .	PATRON_AUTHORIZATION_REQUIRED ,

```
NonEmptyStr ,  
    reason:  
    NonEmptyStr .
```

```
revision.
```

```
VERSION_CONFLIC  
T .
```

La `SourceStatementInput` créée par `RegisterSourceStatement` est une ancre documentaire dans `DceVersion`. Elle ne peut porter ni champ d'interprétation, ni statut d'exigence, ni niveau de conformité. Ces concepts appartiennent au contexte analytique futur `ANA`.

Partie III — Contrats Decision

11. Sous-modèles Decision

Python

```
class DecisionSubjectInput(ApiModel):  
    subject_type: Literal[  
        "CASE", "RISK", "PARTNER", "OFFICIAL_PRICING_VERSION", "SUBMISSION_PACK  
    ]  
    subject_id: UUID  
    case_id: UUID  
    scope_fingerprint: Sha256Hex  
  
class DecisionConditionInput(ApiModel):  
    label: ShortLabel  
    owner_actor_id: UUID  
    due_at: datetime | None = None  
    due_date_absence_reason: NonEmptyStr | None = None  
    failure_consequence: NonEmptyStr  
  
    @field_validator("due_date_absence_reason")  
    @classmethod  
    def due_date_or_reason_is_required(cls, value, info):  
        if info.data.get("due_at") is None and value is None:  
            raise ValueError("due_at or due_date_absence_reason is required")  
        return value  
  
class DecisionContextInput(ApiModel):  
    references: list[VersionedReference] = Field(min_length=1, max_length=250)  
    unknowns: list[NonEmptyStr] = Field(default_factory=list, max_length=100)  
    rationale: NonEmptyStr
```



```
prepared_at: datetime
```

```
class FinalizeDecisionInput(ApiModel):
    decision_id: UUID
    displayed_context_fingerprint: Sha256Hex
    justification: NonEmptyStr

class DecisionConditionEvidenceInput(ApiModel):
    decision_id: UUID
    condition_id: UUID
    evidence_ref: VersionedReference | None = None
    observation: NonEmptyStr
    observed_at: datetime
```

Le `scope_fingerprint` est calculé par le serveur à partir du `CaseScope` retenu. L' API peut accepter une valeur affichée afin de détecter une vue périmée, mais elle ne lui accorde jamais la qualité de vérité.

12. Commandes Decision

command_type / modèle	payload obligatoire	meta	Résultat de succès	Erreurs principales
CreateDecisionDraft Request / CreateDecisionDraft	`decision_type: GO_NO_GO	RISK_ACCEPTANCE	PARTNER_SELECTION	PRICING_APPROVAL
PrepareDecisionContextRequest	decision_id , context: DecisionContextInput .	Existing Decision revision + expected reference revisions.	DECISION_CONTEXT_PREPARED , selected context_fingerprint .	DECISION_CONTEXT_INCOMPLETE , STALE_CONTEXT .
RefreshDecisionContextBeforeFinalizationRequest	decision_id , context: DecisionContextInput , refresh_reason .	Existing Decision revision + expected refs.	DECISION_CONTEXT_REFRESHED , new fingerprint.	DECISION_ALREADY_FINALIZED , STALE_CONTEXT .
ApproveGoDecisionRequest	finalization: FinalizeDecisionInput .	Existing Decision revision + critical expected refs.	GO_DECISION_APPROVED ; Case process may be REFRESH_PENDING .	PATRON_AUTHORIZATION_REQUIRED , DECISION_CONTEXT_FINGERPRINT_MISG .

				MATCH , STALE_CONTEXT .
ApproveConditionalGoDecisionRequest	finalization: FinalizeDecisionInput , conditions: list[DecisionConditionInput] min. 1.	Existing Decision revision + critical expected refs.	CONDITIONAL_GO_DECISION_APPROVED .	CONDITION_OWNERS_REQUIRED , CONDITION_CONSEQUENCE_REQUIRED , STALE_CONTEXT .
ApproveNoGoDecisionRequest	finalization: FinalizeDecisionInput .	Existing Decision revision + critical expected refs.	NO_GO_DECISION_APPROVED .	PATRON_AUTHORIZATION_REQUIRED , DECISION_CONTEXT_FINGERPRINT_MISMATCH .
RecordDecisionConditionSatisfiedRequest	evidence: DecisionConditionEvidenceInput .	Existing Decision revision.	DECISION_CONDITION_SATISFIED .	DECISION_CONDITION_NOT_OPEN , VERSION_CONFLICT .
RecordDecisionConditionFailedRequest	decision_id , condition_id , reason: NonEmptyStr , source_ref optional, observed_at .	Existing Decision revision.	DECISION_CONDITION_FAILED .	DECISION_CONDITION_NOT_OPEN , VERSION_CONFLICT , STALE_CONTEXT .
WaiveDecisionConditionRequest	decision_id , condition_id , justification: NonEmptyStr .	Existing Decision revision.	DECISION_CONDITION_WAIVED .	PATRON_AUTHORIZATION_REQUIRED , DECISION_CONDITION_NOT_OPEN , VERSION_CONFLICT .
MarkDecisionReviewRequiredRequest	Internal only : decision_id , trigger_event_id , reason: NonEmptyStr , changed_reference: VersionedReference .	InternalCommandMeta .	DECISION_MARKED_REVIEW_REQUIRED .	VERSION_CONFLICT .
SupersedeDecisionRequest	decision_id , successor_decision_id , justification: NonEmptyStr .	Existing Decision revision + successor ref.	DECISION_SUPERSEDED .	SUCCESSOR_DECISION_REQUIRED , STALE_CONTEXT .

CancelDecisionDraftRequest	decision_id , reason: NonEmptyStr .	Existing Decision revision.	DECISION_DRAFT_CANCELLED .	DECISION_ALREADY_FINALIZED , VERSION_CONFLICT .
----------------------------	---	--------------------------------	----------------------------	--

Le système refuse tout outcome hors DecisionType. Par exemple, ApproveGoDecision ne peut finaliser qu' une Decision GO_NO_GO ; la validation de prix utilise une Decision PRICING_APPROVAL mais appartient à son slice de prix privé lorsque celui-ci sera activé.

Partie IV — Exemples canoniques et traçabilité

13. Exemple : création de Consultation puis DCE

JSON

```
{
  "meta": {
    "command_id": "b58c09fd-44b0-4e70-8a38-20b0c473bf8a",
    "idempotency_key": "92a86a39-dc0d-4cc1-99f5-3f91f79a1633",
    "correlation_id": "3d44f375-03a0-420b-9bd9-0a6e71fc9fb2"
  },
  "payload": {
    "buyer": {"legal_name": "CANSSM / Filieris", "normalized_identifieur": "CANSSM"},
    "external_reference": "MA26N00017",
    "object": "Travaux de gros œuvre",
    "location": "Guesnain",
    "provenance": {
      "source_channel": "BUYER_PLATFORM",
      "source_reference": "PLACE-ACHETEUR",
      "source_received_at": "2026-08-12T09:00:00+01:00"
    }
  }
}
```

La réponse ne renvoie ni données financières ni documents binaires. Elle renvoie la référence Consultation créée, sa révision, le résultat CONSULTATION_CREATED et l' état de projection.

14. Exemple : approbation Go avec protection contre contexte obsolète

JSON

```
{
  "meta": {
    "command_id": "dc84cd44-742a-46cc-9c36-692d14b6e2c9",
    "idempotency_key": "da8ded5a-0ff5-4ad4-a959-7184d75baad3",
    "correlation_id": "c5340b3e-0c66-4d41-8ff3-f82a409737e3",
    "expected_revision": 4
  },
  "payload": {
    "finalization": {
      "decision_id": "94f18f0d-77d8-4f99-acdc-b57fd393b2d8",
      "displayed_context_fingerprint": "1d49b817c48f82a247ef829e5f8c3f2ef2e6230",
      "justification": "Périmètre maîtrisé et exigences identifiées."
    }
  }
}
```

Si une nouvelle DceVersion, un impact pertinent ou une référence critique a changé, la réponse est un 409 STALE_CONTEXT . Le serveur ne fusionne jamais l’ ancien contexte avec le nouveau à la place du patron.

15. Table de traçabilité APP-01 → DOMAIN-03

APP-01	DOMAIN-03 source	Garantie conservée
PublicCommandMeta , ExistingAggregateCommandMeta	§ 2.2, règles transactionnelles	Idempotence et révision attendue.
VersionedReference	Case § 3.6, DceVersion § 5.5, Decision § 6.7	Lecture de références critiques sans écriture inter-root.
Commandes Case § 7	Case § 3.4 et invariants CASE-INV-*	Cycle commercial et scope non ambigu.
Commandes Consultation § 9	Consultation § 4.2 / CONS-INV-*	Identité consultation séparée de l’ affaire.
Commandes DceVersion § 10	DceVersion § 5.3 / DCE-INV-*	Immutabilité corpus, sources localisées, supersession non destructive.

Commandes Decision § 12	Decision § 6.4 / DEC-INV-*	Patron habilité, contexte immuable, fingerprint, conditions et supersession.
CommandSuccess / CommandProcessing	DOMAIN-03 § 2.3 et § 7	RYOW honnête, processus aval corrélé.
Codes d' erreur § 5	DOMAIN-03 § 3.6, § 5.5, § 6.7	Conflits explicables sans fuite tenant.

16. Décisions de gel APP-01

1. Les routes FastAPI du premier slice exposent un modèle de requête Pydantic fermé par commande ; aucun `payload: dict[str, Any]` public n' est autorisé.
2. `tenant_id` , `actor_id` , `role`, `permissions` et `caused_by_event_id` sont exclus des corps de commande publics.
3. Tout root existant muté requiert `expected_revision` ; les références externes critiques sont déclarées dans `expected_reference_revisions` ; toute création/transition/finalisation requiert `idempotency_key` .
4. Les commandes aval ne passent jamais par une route navigateur ; elles utilisent `InternalCommandMeta` et un `Process Manager` déclaré.
5. Les objets d' un autre bounded context ne sont référencés que par UUID, type et révision/hash ; aucun modèle Pydantic ne transporte un aggregate riche externe.
6. Aucun modèle APP-01 ne contient un montant, une marge, un devis ou une donnée de trésorerie.
7. Le code Pydantic est la traduction exacte de ce document ; toute divergence impose une mise à jour d' APP-01, DOMAIN-03 et du test associé.

Références internes

- `SMART_AO_V8_DOMAIN_01_AGGREGATE_OWNERSHIP_MATRIX.md` — DOMAIN-01 v1.1.
- `SMART_AO_V8_DOMAIN_03_STATE_MACHINES_INVARIANTS_FIRST_SLICE.md` — DOMAIN-03 v1.0.
- `SMART_AO_V8_SPEC_COMMANDES_IDEMPOTENCE.md` — conventions V8 d' idempotence.
- `SMART_AO_V8_DOCUMENTATION_MAP.md` — point de bascule documentaire et roadmap.